CTO, HealthSphere

Proposed STLC Strategy for Consistent, High-Quality Releases.

Testing is not a phase that happens after code is written, it is a risk-control mechanism embedded throughout the SDLC. For **HealthSphere**, where defects can affect clinical operations and patient safety, early and continuous testing is essential. Catching issues at the unit or integration level is exponentially cheaper and safer than discovering them in production. A structured STLC reduces failures by making quality predictable, measurable, and repeatable.

Testing Levels: We should define clear ownership and purpose for each level. **Unit tests** (developers) validate individual functions and build confidence in code correctness. **Integration tests** verify that modules, APIs, and data flows interact correctly for our service-to-database and third-party integrations. **System tests** validate the complete application against requirements in a production-like environment. **Acceptance tests** confirm clinical workflows meet user and regulatory expectations. **Regression tests** protect existing functionality from new changes, and **end-to-end tests** validate real user journeys across the full stack. Each level addresses distinct risks, so coverage becomes deliberate rather than ad hoc.

Test Planning & Design: Every release should begin with a concise test plan defining scope (in/out), risks, environments, and entry/exit criteria. We should estimate effort using historical data and risk-based prioritization, high-risk clinical paths first. Test cases must be clear, traceable to requirements, and include preconditions, steps, expected results, and test data. Peer review of test cases will improve consistency and catch gaps early.

Defect Management: Standardize bug reports with title, environment, steps to reproduce, expected vs. actual results, severity, priority, and evidence (logs, screenshots). Enforce a defined lifecycle (New → Triaged → Assigned → In Progress → Fixed → Verified → Closed) and a triage cadence with dev and product leads. Clearly separate severity (technical impact) from priority (business urgency), a cosmetic issue on a medication screen may be low severity but high priority.

Debugging Principles: Encourage systematic information gathering: reproduce reliably, capture logs and environment details, read stack traces from the topmost application frame, and isolate variables through binary search or logging. Pair debugging and document root causes to prevent recurrence.

STLC Integration: These activities form a continuous loop: planning defines scope, design creates traceable tests, execution generates evidence, defect management drives fixes, and regression closes the loop before release. With clear entry/exit criteria and metrics (defect escape rate, coverage, triage turnaround), **HealthSphere** can shift from reactive firefighting to predictable, high-quality releases.